- **Internship Name : Data Analysis**
- **Task Name: Task 11: Combining Datasets in Pandas using merge(), join() and concat()**
- **Your Name : Shubhangi Naresh Khonde**
- **Internship Organization :**
- **Date:**

1. Objective

To practice combining two related datasets in Pandas using merge(), join() and concat(), and to understand when each function should be used.

2. Tools Used

- **Python 3**
- **Pandas library**
- **VS Code (Jupyter Notebook)**
- **GitHub (for submission)**

3. Datasets

Two related datasets were created, linked by the common column customer_id.

customers (5 rows)

```python
import pandas as pd

customers = pd.DataFrame({
    'customer_id': [1, 2, 3, 4, 5],
    'name': ['Amit', 'Priya', 'Rahul', 'Sneha', 'Rohan'],
    'city': ['Nagpur', 'Pune', 'Mumbai', 'Nagpur', 'Nashik']
})

orders = pd.DataFrame({
    'order_id': [101, 102, 103, 104, 105, 106],
    'customer_id': [1, 2, 2, 6, 3, 1],
    'amount': [500, 1200, 300, 800, 950, 400]
})

print(customers)
print(orders)
```

```
   customer_id   name    city
0            1   Amit  Nagpur
1            2  Priya    Pune
2            3  Rahul  Mumbai
3            4  Sneha  Nagpur
4            5  Rohan  Nashik
   order_id  customer_id  amount
0       101            1     500
1       102            2    1200
2       103            2     300
3       104            6     800
4       105            3     950
5       106            1     400
```

Note: Customer 6 appears in orders but not in customers. Customers 4 and 5 have no orders. This was done deliberately to show how each join type behaves.

4. Merge()

merge() combines two DataFrames based on the values of a common column, similar to a SQL JOIN.

python

pd.merge(customers, orders, on='customer_id', how='inner')

Type	Behavior	Result
inner	Keeps only matching rows	5 rows (customers 1, 2, 3)
left	Keeps all customers; fills NaN where no order exists	7 rows (Sneha and Rohan have NaN)
right	Keeps all orders; fills NaN where no customer exists	6 rows (order 104 has NaN name and city)
outer	Keeps all rows from both tables	8 rows

Using indicator=True in the outer merge adds a _merge column showing the source of each row: both, left_only or right_only. Customers 4 and 5 were left_only, and order 104 was right_only.

Observation: The order_id and amount columns became decimals (101.0, 500.0) in the left and outer merges because NaN values force Pandas to use the float type.

```
##Step 2: merge() ( SAME as SQL JOIN )
```

```
# Inner: fakt matching rows
inner = pd.merge(customers, orders, on='customer_id', how='inner')
print(inner, sep='\n\n')
```

	customer_id	name	city	order_id	amount
0	1	Amit	Nagpur	101	500
1	1	Amit	Nagpur	106	400
2	2	Priya	Pune	102	1200
3	2	Priya	Pune	103	300
4	3	Rahul	Mumbai	105	950

```
# Left: sagle customers, order nasel tar NaN
left = pd.merge(customers, orders, on='customer_id', how='left')
print(left, sep='\n\n')
```

	customer_id	name	city	order_id	amount
0	1	Amit	Nagpur	101.0	500.0
1	1	Amit	Nagpur	106.0	400.0
2	2	Priya	Pune	102.0	1200.0
3	2	Priya	Pune	103.0	300.0
4	3	Rahul	Mumbai	105.0	950.0
5	4	Sneha	Nagpur	NaN	NaN
6	5	Rohan	Nashik	NaN	NaN

```
# Right: sagle orders
right = pd.merge(customers, orders, on='customer_id', how='right')
print(right, sep='\n\n')
```

	customer_id	name	city	order_id	amount
0	1	Amit	Nagpur	101	500
1	2	Priya	Pune	102	1200
2	2	Priya	Pune	103	300
3	6	NaN	NaN	104	800
4	3	Rahul	Mumbai	105	950
5	1	Amit	Nagpur	106	400

```
# Outer: donhi kadche sagle rows
outer = pd.merge(customers, orders, on='customer_id', how='outer', indicator=True)

print(outer, sep='\n\n')
```

	customer_id	name	city	order_id	amount	_merge
0	1	Amit	Nagpur	101.0	500.0	both
1	1	Amit	Nagpur	106.0	400.0	both
2	2	Priya	Pune	102.0	1200.0	both
3	2	Priya	Pune	103.0	300.0	both
4	3	Rahul	Mumbai	105.0	950.0	both
5	4	Sneha	Nagpur	NaN	NaN	left_only
6	5	Rohan	Nashik	NaN	NaN	left_only
7	6	NaN	NaN	104.0	800.0	right_only

5. Join()

join() combines DataFrames using the **index**. The customer_id column was first set as the index.

python

```
cust_idx = customers.set_index('customer_id')
ord_idx = orders.set_index('customer_id')
joined = cust_idx.join(ord_idx, how='left')
```

- The left join kept all 5 customers, and Sneha and Rohan got NaN for order details.
- The inner join with lsuffix and rsuffix kept only customers who have orders.
- The suffix parameters are used to avoid conflicts when both DataFrames have columns with the same name.

```
##Step 3: join() (index based)
```

```
cust_idx = customers.set_index('customer_id')
ord_idx = orders.set_index('customer_id')

joined = cust_idx.join(ord_idx, how='left')
print(joined)
```

```
             name    city  order_id  amount
customer_id
1            Amit  Nagpur     101.0   500.0
1            Amit  Nagpur     106.0   400.0
2           Priya    Pune     102.0  1200.0
2           Priya    Pune     103.0   300.0
3           Rahul  Mumbai     105.0   950.0
4           Sneha  Nagpur       NaN     NaN
5           Rohan  Nashik       NaN     NaN
```

```
cust_idx.join(ord_idx, how='inner', lsuffix='_cust', rsuffix='_ord')
```

customer_id	name	city	order_id	amount
1	Amit	Nagpur	101	500
1	Amit	Nagpur	106	400
2	Priya	Pune	102	1200
2	Priya	Pune	103	300
3	Rahul	Mumbai	105	950

6. concat()

concat() does not match on values. It simply stacks DataFrames together.

a) Vertical (axis=0): Two new customers (Neha, Karan) were added below the existing customers using ignore_index=True. The result had 7 rows with a fresh index from 0 to 6.

```
##Step 4: concat() (stack)
##a Vertical (rows khali add)
```

```
new_customers = pd.DataFrame({
    'customer_id': [6, 7],
    'name': ['Neha', 'Karan'],
    'city': ['Delhi', 'Nagpur']
})

all_customers = pd.concat([customers, new_customers], ignore_index=True)
print(all_customers)
```

[13]

```
   customer_id   name    city
0            1   Amit  Nagpur
1            2  Priya    Pune
2            3  Rahul  Mumbai
3            4  Sneha  Nagpur
4            5  Rohan  Nashik
5            6   Neha   Delhi
6            7  Karan  Nagpur
```

b) **Horizontal (axis=1):** customers and orders were placed side by side. Rows were aligned by **index position**, not by customer_id. Because orders has 6 rows and customers has 5, the sixth row shows NaN for the customer columns. The customer_id values in the two halves do not correspond, which shows that horizontal concat is not suitable for relational joining.

```
##b Horizontal (columns shejari)
```

```
side_by_side = pd.concat([customers, orders], axis=1)
print(side_by_side)
```

```
   customer_id   name    city  order_id  customer_id  amount
0          1.0   Amit  Nagpur       101            1     500
1          2.0  Priya    Pune       102            2    1200
2          3.0  Rahul  Mumbai       103            2     300
3          4.0  Sneha  Nagpur       104            6     800
4          5.0  Rohan  Nashik       105            3     950
5          NaN    NaN     NaN       106            1     400
```

c) **Using keys:** keys=['old', 'new'] created a hierarchical index that labels which rows came from which DataFrame.

```
##(c) keys vapra (konta data konkadcha kalayla)
```

```
pd.concat([customers, new_customers], keys=['old', 'new'])
```

		customer_id	name	city
old	0	1	Amit	Nagpur
	1	2	Priya	Pune
	2	3	Rahul	Mumbai
	3	4	Sneha	Nagpur
	4	5	Rohan	Nashik
new	0	6	Neha	Delhi
	1	7	Karan	Nagpur

7. Extra Analysis

After an inner merge, the total order amount per city was calculated:

python

```
merged.groupby('city')['amount'].sum()
```

Customers who never placed an order were identified using a left merge with indicator=True, then filtering _merge == 'left_only'. The result was **Sneha** and **Rohan**.

City	Total Amount
Mumbai	950
Nagpur	900
Pune	1500

```
##Step 6: Extra practice
```

```
# Merge nantar analysis
merged = pd.merge(customers, orders, on='customer_id', how='inner')

# Total amount per city
print(merged.groupby('city')['amount'].sum())

# Konte customers ni kahich order kele nahit?
no_orders = pd.merge(customers, orders, on='customer_id', how='left', indicator=True)
print(no_orders[no_orders['_merge'] == 'left_only'])
```

```
city
Mumbai     950
Nagpur     900
Pune      1500
Name: amount, dtype: int64
   customer_id   name    city  order_id  amount     _merge
5            4  Sneha  Nagpur       NaN     NaN  left_only
6            5  Rohan  Nashik       NaN     NaN  left_only
```

8. Comparison

Feature	merge()	join()	concat()
Combines based on	Column values	Index	Position / axis
Join types	inner, left, right, outer	left (default), right, inner, outer	Stacks only
Best used for	SQL-style joins on a key column	Quick joins when the index is already set	Appending rows or placing columns side by side
Handles unmatched rows	Yes (NaN)	Yes (NaN)	Aligns by index, may create NaN

9. Conclusion

- merge() is the most flexible and is best when two tables share a key column.
- join() is a shorter syntax for joining on the index.
- concat() is best for appending datasets with the same structure (for example, new rows) and not for relational matching.
- Choosing the right join type (inner, left, right, outer) depends on whether unmatched rows should be kept or dropped.
- Using indicator=True is very helpful for finding unmatched records.